

ParasGB: A Graph Benchmark Suite for Parasitic Estimation on AMS Circuits

Jiajun Zou¹, Jiawei Liu², Ao Liu¹, Junnong Tian¹, Yibin Zhang³, Chengjie Liu^{4,5}, Yuxi Wang⁴,
Shan Shen^{1*}, Wenhua Gu^{1*}, Jun Yang^{5,6}, and Wenjian Yu^{3*}

¹Nanjing University of Science and Technology, Nanjing, China

²The Chinese University of Hong Kong, Hong Kong SAR, China

³Dept. Computer Science & Tech., BNRist, Tsinghua University, Beijing, China

⁴School of Electronic Science and Engineering, Nanjing University, Nanjing, China

⁵National Center of Technology Innovation for EDA, Nanjing, China

⁶School of Integrated Circuits, Southeast University, Nanjing, China

Abstract

As chip manufacturing processes advance to deep submicron nodes, parasitic interconnect effects increasingly dominate the performance of analog and mixed-signal (AMS) circuits and often lead to costly layout iterations. This makes early-stage estimation of parasitic capacitance and resistance important for parasitic-aware design exploration before full physical implementation. However, progress on GNN-based parasitic modeling has been hindered by the lack of public, high-fidelity RC benchmarks that support reproducible evaluation. To address this gap, we introduce ParasGB, the first open-source benchmark suite for pre-layout parasitic parameter prediction on circuit graphs. ParasGB provides large-scale, heterogeneous RC networks extracted with commercial EDA tools from tape-out-proven designs, together with a unified evaluation protocol covering node-level ground capacitance, edge-level resistance, and edge-level coupling capacitance. Within this framework, we benchmark diverse GNN architectures using a standardized training pipeline and expose challenges such as extreme label imbalance, long-tailed parasitic distributions, and strong structural heterogeneity. By establishing a physically grounded and standardized benchmark for early-stage parasitic prediction, ParasGB provides an open platform for reproducible research on circuit graph learning and parasitic-aware model development. All datasets, pre-processing scripts, and configurations are publicly available in our code repository.¹

1 Introduction

As chip manufacturing advances toward process nodes of 7nm and below, and as 3D integrated-circuit (IC) technologies continue to emerge [31], parasitic effects, i.e., interconnect resistance (R) and capacitance (C), have shifted from a secondary consideration to a key bottleneck that constrains design convergence. At these process nodes, the interconnect latency has significantly exceeded the intrinsic latency of the device. For example, in a 5nm node, it accounts for even 70%–80% of the total delay of the path [40]. These effects are particularly severe in analog/mixed-signal (AMS) circuits and memory circuits, where weaker device drive capability at advanced nodes, together with increasingly complex multi-layer metal stacks

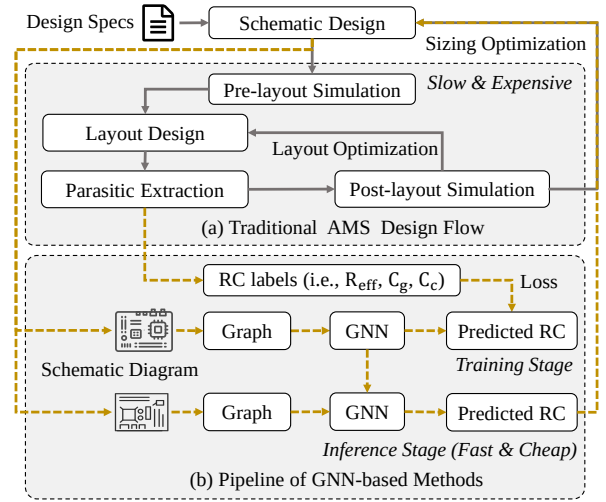


Figure 1: Comparison between traditional AMS design flow and parasitic-aware flow integrating GNNs. Estimating parasitic effects in the early stage can reduce the number of design iterations and help the design converge.

and abundant vias, further amplifies interconnect RC effects. As a result, parasitics no longer merely perturb circuit behavior, but significantly increase design iterations and make front-end/back-end design convergence substantially more difficult.

As shown in Figure 1 (a), the typical design process for analog circuits and memory circuits (e.g., static random access memory, SRAM) starts with schematic design and then moves on to the time-consuming and laborious layout phase. Unlike digital circuit design, the layout design of analog and memory circuits is mostly done manually, which often takes months. Crucially, the impact of parasitic resistance and capacitance (RC) can only be accurately captured through post-layout extraction. This dependency creates a significant design bottleneck: if post-layout simulation fails to meet performance metrics due to unpredictable parasitic effects, designers have to revert to the layout design stage or even restart the schematic design work, resulting in costly and time-consuming iterations. Therefore, the core goal of early parasitic prediction is not simply to accelerate the simulation process or deal with the

¹<https://github.com/ShenShan123/ParasGB.git>

This work was supported by the Fundamental Research Funds for the Central Universities (Grant No. 30925010605), Jiangsu Provincial Key Research and Development Program (No. BE2023818), Basic Research Program of Jiangsu (Nos. BK20243046, BK20253062). *Corresponding authors: S. Shen, W. Gu, and W. Yu.

Table 1: Comparison of circuit datasets and benchmarks. Existing benchmarks focus on digital congestion/timing or schematic-level analog tasks. ParasGB provides RC labels for post-layout parasitic estimation on both analog and SRAM designs.

Dataset	Domain	Design Stage	Key Tasks	RC Included
CircuitNet [1]	Digital	Schematic+Layout	Congestion/Timing	×
EDA-schema [39]	Digital	Schematic+Layout	Representation	×
CktGNN [5]	Analog	Schematic	Topology/Sizing	×
AMSNet [43]	Analog	Schematic	Generation/Understanding	×
AnalogGenie [8]	Analog	Schematic	Topology Generation/Discovery	×
ParasGB (Ours)	Analog/Memory	Schematic	Parasitic Estimation	✓

complexity of large-scale networks, but to provide actionable parasitic guidance before layout is finalized. For front-end designers, such guidance can help estimate parasitics on critical paths and support decisions on driver sizing, buffer insertion, and device dimension tuning. For back-end designers, it can help identify critical nets, guide floorplanning (e.g., symmetry-aware placement requirements), and assist routing decisions such as wire width allocation and metal layer selection. By introducing parasitic parameter-aware design concepts early in the design process, this technology can effectively reduce the number of design iterations and greatly shorten the overall chip development cycle [32, 37].

In recent years, graph-based learning, especially graph neural networks (GNNs), has emerged as a promising direction for RC parasitic modeling [32, 33, 37], as illustrated in Figure 1(b). Existing studies span schematic-based parasitic prediction, layout-aware circuit representation learning [52], and digital circuit RC prediction [49], indicating growing interest in parasitic-aware learning across different circuit domains and design stages. These efforts also suggest that early parasitic prediction can benefit downstream optimization and analysis tasks, such as parasitic aware design tuning and accelerated post-layout simulation [23, 34]. We defer a more detailed discussion of these methods to Section 2.2.

Despite this progress, research in this direction remains limited by the lack of a unified benchmark system, and there is still no high-quality public dataset or widely recognized evaluation standard comparable to the Open Graph Benchmark (OGB) [12]. Due to intellectual-property restrictions and the high cost of industrial-grade IC design, open-source research often relies on outdated or small-scale cases, forming isolated data islands that hinder objective comparison across algorithms and limit the applicability of existing methods to real engineering scenarios.

Beyond its value for EDA research, ParasGB also provides a challenging testbed for graph learning, with large-scale, heterogeneous, and physically grounded circuit graphs derived from silicon-proven designs. It can support future research on more effective and targeted graph foundation models for circuit-specific tasks. Our core contributions are summarized as follows:

- This paper publishes a high-quality dataset derived from the AMS circuits designed with commercial EDA tools, providing gold standard parasitics, and laying a reliable foundation for related research.

- We construct a comprehensive benchmark suite for pre-layout parasitic prediction, covering node-level ground capacitance prediction and edge-level effective resistance and coupling capacitance prediction under both regression and classification settings.
- This paper develops an easy-to-use research framework that references the OGB design style and integrates data loading and automatic scoring functions, so that researchers familiar with PyTorch Geometric can quickly get started with experiments.

2 Related Work

In this section, we review prior work on EDA benchmarks and GNN-based parasitic estimation. We first discuss existing machine-learning benchmarks in EDA, and then summarize representative graph-learning methods for parasitic prediction and related tasks.

2.1 Machine Learning for EDA Benchmarks

In recent years, machine learning (ML) has been widely adopted in electronic design automation (EDA), and standardized benchmarks have become increasingly important for ensuring fair comparison and reproducible research. However, most existing benchmarks remain confined to specific design stages and circuit types, as summarized in Table 1. For digital circuits, CircuitNet [1, 14] supports large-scale evaluation on representative tasks such as congestion prediction and timing prediction, while EDA schema [39] proposes a unified graph data model to standardize circuit representations across design stages. In contrast, analog circuits follow substantially different design flows and optimization objectives, which motivates dedicated benchmarks in this domain. Early efforts such as CktGNN [5] focus on schematic-level topology generation and device-parameter tuning, while AMSNet [38, 43] and AnalogGenie [8] support analog circuit generation and understanding at the netlist level. Nevertheless, these benchmarks predominantly remain at the schematic level and do not provide post-layout parasitic RC supervision for analog and memory circuits. This gap motivates ParasGB as a standardized benchmark for parasitic prediction on physically grounded circuit graphs.

2.2 Parasitic Estimation via GNNs

Parasitic parameter estimation is a critical task in EDA, especially at advanced technology nodes where interconnect effects strongly influence circuit performance. Since circuits naturally form graph-structured data, GNNs have become a promising tool for learning

Table 2: Analog Circuit Dataset Statistics. Topology edges ($dev2pin$, $pin2net$) form the input graph, while labeled targets are effective resistance (R_{eff}) and ground capacitance (C_g).

Scale	ID	Type	Nodes				Edges			C_g Statistics			R_{eff} Statistics			
			total	device	pin	net	total	$pin2net$	$dev2pin$	n_{mean}	n_{min}	n_{max}	total	e_{mean}	e_{min}	e_{max}
XS	19	BGR [13]	231	68	156	7	428	156	272	2.70E-14	8.28E-15	6.61E-14	5166	50.21	0.23	287.26
	13	OP [41]	256	66	180	10	444	180	264	2.99E-14	5.45E-15	7.64E-14	3576	34.14	0.01	205.30
	4	BGR [18]	448	132	306	10	834	306	528	1.78E-13	1.41E-14	3.41E-13	12170	74.65	0.79	590.21
	9	LDO [26]	1541	478	1048	15	2960	1048	1912	2.56E-13	1.62E-14	1.22E-12	26796	70.16	0.01	349.64
	10	OP [28]	1298	408	873	17	2501	873	1628	8.31E-14	2.81E-15	4.24E-13	21294	41.58	0.13	615.11
S	11	LDO [21]	2899	902	1978	19	5586	1978	3608	3.03E-13	2.08E-14	1.01E-12	33078	70.24	0.00	529.63
	16	LDO [45]	1364	413	924	27	2550	924	1626	1.65E-13	1.04E-15	8.08E-13	19554	133.33	0.01	526.15
	2	OP [46]	534	152	350	32	914	350	564	6.58E-14	1.07E-15	5.16E-13	13774	65.72	0.01	756.66
	1	LVDS [20]	1100	251	814	35	1776	814	962	5.22E-14	1.04E-15	3.01E-13	21554	63.42	0.01	264.93
	12	LDO [16]	1409	440	930	39	2634	930	1704	5.80E-14	1.04E-15	5.82E-13	15212	58.69	0.01	294.71
	6	LDO [10]	1836	553	1242	41	3414	1242	2172	2.18E-13	1.04E-15	1.85E-12	29740	133.82	0.01	512.75
	3	BGR [2]	1225	382	800	43	2260	800	1460	5.60E-14	1.04E-15	8.97E-13	11128	20.43	0.71	235.77
	17	LDO [22]	1451	438	961	52	2637	961	1676	1.09E-13	1.04E-15	1.70E-12	22054	405.55	0.01	3202.96
	8	LDO [42]	1566	475	1027	64	2823	1027	1796	6.20E-14	1.06E-15	9.54E-13	20828	88.09	0.01	350.96
M	14	BGR [30]	591	138	362	91	746	362	384	1.13E-14	1.03E-15	2.94E-13	6756	134.92	0.06	386.95
	18	LDO [29]	2496	758	1627	111	4467	1627	2840	6.64E-14	1.05E-15	1.35E-12	24972	57.12	0.02	289.77
	15	LDO [11]	3672	1020	2235	417	5535	2235	3300	3.07E-14	1.03E-15	2.14E-12	38322	60.79	0.01	326.16
	20	BGR [6]	2388	621	1304	463	2871	1304	1567	7.16E-15	1.04E-15	5.58E-13	14874	41.76	0.00	559.43
	5	LDO [19]	6917	1738	3496	1683	7092	3496	3596	2.18E-15	1.04E-15	2.76E-13	6512	171.95	0.90	1972.48
	7	LDO [19]	6917	1738	3496	1683	7092	3496	3596	2.18E-15	1.04E-15	2.76E-13	6512	171.95	0.90	1972.48
Total Average			2007.0	558.6	1205.5	243.0	2978.3	1205.5	1772.8	8.91E-14	4.11E-15	7.82E-13	17693.6	97.4	0.2	711.5

Table 3: SRAM Circuit Dataset Statistics. Topology edges ($dev2pin$, $pin2net$) form the input graph; parasitic targets include node-level ground capacitance C_g , aggregated coupling-capacitance edges (C_c) and resistance edges R_{eff} .

Scale	Dataset	Nodes				Edges			C_g Statistics			C_c Statistics				R_{eff} Statistics			
		total	device	pin	net	total	$dev2pin$	$pin2net$	n_{mean}	n_{min}	n_{max}	total	e_{mean}	e_{min}	e_{max}	total	e_{mean}	e_{min}	e_{max}
L	digtime [35]	17.6K	4.1K	12.4K	1.0K	24.8K	12.4K	12.4K	3.17E-17	1.14E-19	8.63E-15	44.4K	8.03E-18	3.10E-25	4.62E-14	99.8K	427.40	2.58	3009.74
	timing_ctrl	18.2K	4.4K	13.1K	668	26.2K	13.1K	13.1K	1.42E-16	3.31E-20	4.13E-15	43.1K	2.19E-17	4.44E-27	3.86E-14	99.6K	212.88	4.97	1168.11
XL	array_128	143K	32.8K	98.3K	12.6K	197K	98.3K	98.3K	1.19E-16	6.66E-19	5.98E-14	414K	7.32E-18	5.65E-27	6.96E-14	100K	232.61	9.34	1198.62
	sram	249K	57.4K	172K	19.9K	344K	172K	172K	5.48E-17	1.55E-25	1.43E-14	760K	8.30E-18	1.37E-26	3.14E-13	41.9K	266.70	22.04	413.77
XXL	ultra8t [36]	10.1M	2.3M	6.9M	861K	13.8M	6.9M	6.9M	1.60E-16	1.41E-23	9.46E-12	33.6M	1.03E-17	9.02E-38	1.84E-11	437K	87.65	8.49	183.84
	sandwich [48]	11.7M	2.6M	7.9M	1.1M	15.8M	7.9M	7.9M	2.46E-16	1.11E-25	9.10E-12	38.4M	1.13E-17	4.78E-34	2.08E-11	1.0M	286.03	22.10	1203.66
Total Average		3.70M	833K	2.52M	332K	5.04M	2.52M	2.52M	1.26E-16	1.36E-19	3.10E-12	12.2M	1.12E-17	5.17E-26	6.53E-12	296K	252.21	11.59	1196.29

the relationship between circuit topology, device features, and parasitic behavior. Compared with rule-based estimation or heuristic methods, GNN-based approaches provide a data-driven alternative for capturing complex parasitic patterns across different circuit structures.

ParaGraph [32] pioneered the use of GNNs for parasitic-aware circuit learning by predicting layout parasitics and device parameters directly from circuit graphs. More recent work has extended this direction to different settings and targets. CircuitGCL [33] improves representation learning for parasitic prediction through contrastive learning, while CircuitGPS [37] addresses coupling capacitance prediction in few-shot scenarios. In the digital domain, ParaFormer [49] combines heterogeneous GNNs with transformer-style modeling for pre-routing parasitic RC prediction, highlighting the importance of jointly modeling circuit topology and geometric relationships. Related work, such as TAG [52] further suggests that layout-aware circuit representations can also benefit parasitic-related downstream tasks. Beyond direct parasitic prediction, predicted parasitic quantities have also been used in downstream tasks

such as yield evaluation and parasitic-aware optimization, further demonstrating their practical value [23, 34]. Despite this progress, existing studies remain fragmented in datasets, task definitions, and evaluation protocols, making fair and reproducible comparison difficult. This motivates the development of ParasGB, which provides a unified benchmark for pre-layout parasitic prediction with standardized datasets, tasks, and evaluators.

3 The ParasGB Benchmark

In this section, we introduce ParasGB as a benchmark derived from real industrial circuit design flows. We summarize its data characteristics, describe its graph construction principles, and present the evaluation setting for fair and reproducible comparison.

3.1 Dataset Statistics

Dataset scale and label distribution are fundamental to benchmarking realistic parasitic prediction workloads. ParasGB therefore includes both analog circuits and SRAM circuits, covering a broad

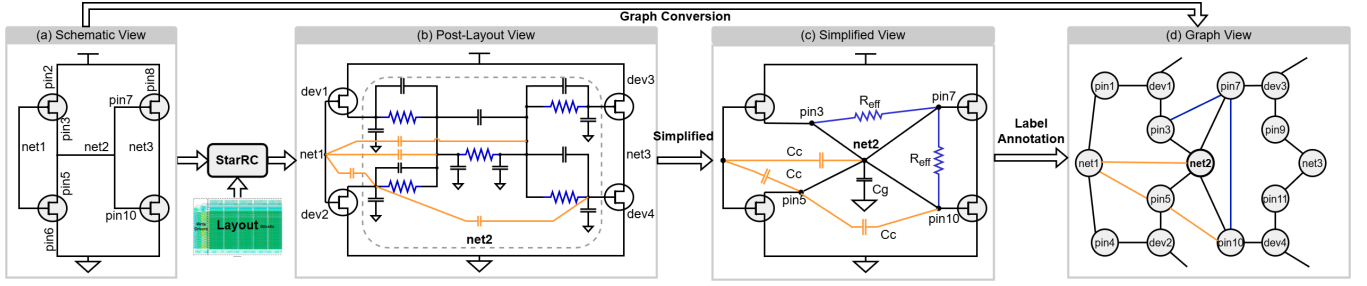


Figure 2: SRAM circuit graph conversion: (a) The schematic. (b) Post-layout netlist with extracted RCs by StarRC. (c) Our simplified netlist with effective RCs. (d) The converted circuit graph.

range of graph sizes and circuit characteristics. Following the number of net nodes, we group analog designs into three scales (XS, S, and M) and SRAM designs into three scales (L, XL, and XXL), since net count largely determines the size of the circuit matrix and thus the computational cost of simulation [15]. The overall statistics are summarized in Table 2 and Table 3, while detailed label histograms for all 20 analog circuits and 6 SRAM subsets are provided in the repository. (details in Section A) The analog subset spans from micro-scale blocks to more complex modules. The smallest circuit contains only 231 nodes, while the largest ones reach 6917 nodes. As reported in Table 2, these designs also vary substantially in the number of components, pins, nets, and parasitic labels. Although the graph sizes are relatively small compared with SRAM circuits, their parasitic values are extremely fine-grained: most capacitances fall in the range of $10^{-14} \sim 10^{-13}$ F. Such tiny magnitudes place strict requirements on the numerical accuracy, stability, and robustness of prediction models.

In contrast, the SRAM subset reaches industrial scale and introduces graphs that are orders of magnitude larger. Small SRAM designs such as *digtime* contain about 1.8×10^4 nodes, while larger designs such as *sandwich* contain up to 1.17×10^7 nodes and 5.52×10^7 edges. As shown in Table 3, SRAM parasitic graphs are also highly dense, with coupling-capacitor edges dominating the edge set in large designs. This makes SRAM subsets particularly challenging not only in terms of graph scale, but also in terms of memory footprint and computational efficiency.

Taken together, the analog and SRAM subsets form a representative cross-scale benchmark. Analog circuits emphasize fine-grained parasitic variation and prediction precision, while SRAM circuits stress scalability on ultra-large and densely connected graphs. This combination makes ParasGB suitable for evaluating both accuracy and efficiency, providing a more comprehensive benchmark than prior datasets limited to a single scale or circuit domain.

3.2 Circuit Sourcing and Validation

All data in ParasGB are derived from real chip design flows and generated with industry-standard commercial EDA toolchains. For the analog subset, the designs are implemented in a 180nm BCD process. In the front-end stage, schematic capture and pre-layout simulation are performed using Cadence Virtuoso together with Spectre. In the back-end stage, physical layouts are created in Virtuoso, followed by DRC/LVS verification and high-accuracy parasitic

extraction using Siemens Calibre xRC. The analog subset covers representative AMS modules, including LVDS (Low-Voltage Differential Signaling), BGR (Band Gap Reference), OP (Operational Amplifier), and LDO (Low Dropout Regulator).

For the SRAM subset, the design flow follows a typical memory implementation and signoff process under the TSMC 28nm process. Starting from verified circuit designs and completed layouts, circuit-level functional and timing verification is performed using HSPICE and FineSim at both the pre-layout and post-layout stages. Post-layout parasitics are then extracted using Synopsys StarRC, a silicon-accurate signoff parasitic extraction tool that is widely used for memory, analog/mixed-signal, and custom digital designs. StarRC is designed to support large memory blocks and advanced process effects while preserving signoff-level accuracy, making it well-suited for generating realistic parasitic annotations for SRAM circuits.

Moreover, all circuit designs in ParasGB are based on real tape-out-proven projects rather than purely theoretical or simulation-only examples, which gives the dataset strong physical fidelity and practical relevance. Under advanced process nodes, layouts become increasingly compact and metal spacing is reduced, which intensifies parasitic coupling and increases the structural complexity of parasitic networks. This is a central challenge in parasitic prediction and one of the key scenarios targeted by ParasGB. By extracting data directly from tape-out-proven designs, ParasGB narrows the gap between idealized academic benchmarks and real industrial conditions, and helps ensure that learned representations better reflect underlying physical mechanisms of circuit parasitics rather than artifacts of overly simplified models. More detailed information about the dataset is provided in our repository.

3.3 Topology-to-Graph Conversion

The transformation from circuit topology to graph representation follows the same overall framework for both SRAM and analog circuits, while the specific supervision labels depend on the task setup of each circuit family. Figure 2 illustrates this process using an SRAM example (analog process is in the Section B). Starting from the schematic and layout, commercial tools generate a post-layout netlist with fully extracted parasitics, which is then simplified and converted into a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. The node set $\mathcal{V}$ contains three types: device nodes representing circuit components (e.g., MOSFETs and resistors), net nodes representing interconnect

Table 4: ParasGB Dataset Splits and Task Granularity.

Category	Graph Size	Design	Graph Task				Parasitic Task			Metric	
			Node-level		Edge-level		C_g	C_c	R_{eff}	Class.	Reg.
			Class.	Reg.	Class.	Reg.					
SRAM	L	digtime, timing_ctrl									
	XL	ssram, array_128_32_8t	✓	✓	✓	✓	✓	✓	✓	acc/ F_1	mae/ R^2
	XXL	ultra8t, sandwich									
Analog	XS	19, 13, 4, 9, 10, 11									
	S	16, 2, 12, 6, 3, 17, 8, 14	✓	✓	✓	✓	✓	—	✓	acc/ F_1	mae/ R^2
	M	18, 15, 20, 5, 7									

Table 5: Definition of Analog Circuit Graph Node Features.

Type	Feature	Definition	Index
Device	D_{mos}	Is MOS device	0
	D_{res}	Is resistor	1
	D_{cap}	Is capacitor	2
	W	Width of the transistor	3
	L	Length of the transistor	4
	M_{mos}	Multiplier of transistors	5
	N_f	Number of fingers	6
	T	Type of the device	7
Net	F_{pwr}	Power Net flag	0
	F_{gnd}	Ground Net flag	1
	F_{sig}	Signal Net flag	2
	F_{io}	Is connected to external I/O port	3
	N_{mos}	Number of connected transistors	4
	N_g	Number of connected gate terminals	5
	N_{sd}	Number of connected source/drain terminals	6
	N_b	Number of connected base terminals	7
	L_{tot}	Total channel length of connected transistor	8
	W_{tot}	Total channel width of connected transistor	9
Pin	P_d	Is connected to drain terminal	0
	P_g	Is connected to gate terminal	1
	P_s	Is connected to source terminal	2
	P_b	Is connected to bulk terminal	3
	P_{cap}	Is connected to capacitor terminal	4
	P_{res}	Is connected to resistor terminal	5

wires, and pin nodes representing device terminals. Topological edges $\mathcal{E}_{topo}$ (black lines in the figure) capture schematic connectivity through *device-to-pin* and *pin-to-net* relations and are shared by both SRAM and analog circuits. Among parasitic labels, blue edges denote effective resistance (R_{eff}), which is used in both circuit families, whereas yellow edges denote coupling capacitance (C_c), which is included only in the SRAM benchmark. For completeness, we provide an analog-specific graph construction pipeline in the repository.

Converting post-layout netlists into lumped models is necessary because raw parasitic extraction results are prohibitively complex. As shown in Figure 2 (b), even a single logical net can expand into a large distributed RC network with a huge number of parasitic components, making direct learning intractable. We therefore simplify each distributed network into a lumped circuit model that defines our supervision signals. Specifically, *net nodes* are labeled with lumped ground capacitance (C_g), while labeled edges $\mathcal{E}_{label}$

comprise effective resistance (R_{eff}), computed for sampled pin pairs within the same resistive network using a matrix-based method derived from the node admittance matrix and the inverse of its Cholesky factor [24] (The detailed algorithm is in Section E). In addition, for SRAM circuits, coupling edges (C_c) aggregate the total coupling capacitance between corresponding net pairs. The ground and coupling capacitances mentioned above both originate from unintended electric-field interactions in layout structures: ground capacitance is formed between interconnects and the substrate, whereas coupling capacitance is formed between adjacent interconnect segments. This reduction preserves the key timing- and noise-related characteristics of the original RC network while making graph-based learning on large industrial designs computationally feasible.

3.4 Node Feature Engineering

The core objective of feature engineering is to turn the schematic design into learnable numerical representations that approximate unobserved layout-dependent parasitics. Statistics extracted from schematic netlists act as high-level descriptors of the underlying physical topology. For instance, the number and type of devices attached to a net (node degree) serve as proxies for interconnect length and via count: higher connectivity typically implies longer wiring, more contacts, and thus larger parasitic resistance and capacitance. Likewise, device geometric parameters such as transistor width (W), length (L), and multiplier (M_{mos}) directly constrain the size of active regions and poly gates. Increasing W or M_{mos} raises intrinsic gate and diffusion capacitance, and also forces wider and denser metal routing, which introduces additional parasitic resistance and capacitance in the metal interconnect. These relationships establish a meaningful predictive correlation between schematic-level attributes and post-layout RC values.

Guided by this intuition, we encode analog circuits using a compact hierarchical feature set at the device, net, and pin levels (see Table 5). Device features combine type indicators (D_{mos} , D_{res} , D_{cap}) with key geometric and process parameters (W , L , M_{mos} , N_f , T). Net features include functional flags (F_{pwr} , F_{gnd} , F_{sig} , F_{io}) and connectivity statistics (N_{mos} , N_g , N_{sd} , N_b , L_{tot} , W_{tot}) summarizing how many devices and terminals a net drives and the total channel dimensions it aggregates. Pin features (P_d , P_g , P_s , P_b , P_{cap} , P_{res}) indicate each terminal’s electrical role and local loading context. SRAM circuits share the same hierarchical feature-design principle and high-level

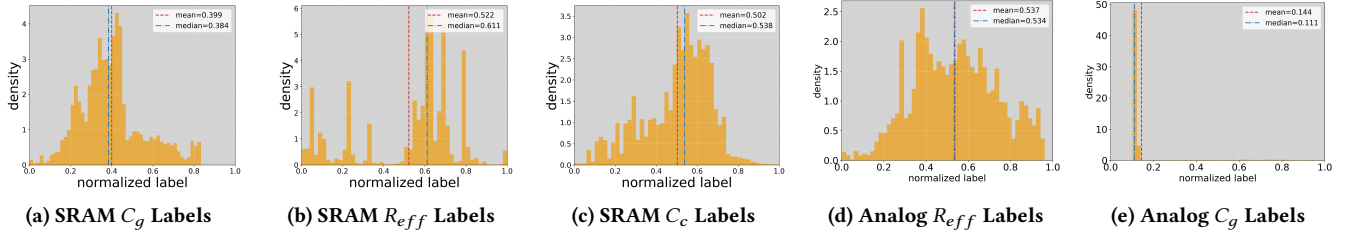


Figure 3: Label Distributions for ParasGB-SRAM and ParasGB-Analog. The pronounced data imbalance poses significant challenges for GNN training, often leading models to overfit dominant samples while neglecting critical outliers.

node-feature categories as analog circuits, namely device-, net-, and pin-level features. However, because the underlying circuit structures are different, some feature dimensions have slightly different physical meanings and are adjusted to better reflect memory-netlist characteristics. For example, pin features in analog circuits mainly characterize connection roles and electrical functions, whereas pin features in SRAM circuits primarily encode MOS terminal types such as G/D/S/B (details in Appendix A).

3.5 Benchmarking Baselines

Basic GNN Baselines. In order to verify the task challenge of the ParasGB dataset, this paper selects the classical model in the graph learning field as the basic benchmark. GCN [17], GAT [44], GraphSAGE [9], and PNA [3] are the mainstream methods in the field of GNNs, which learn the topological features of circuits through the message passing mechanism between neighbor nodes. In addition, this paper also incorporates graph transformer models such as SGFormer [47] and PolyNormer [4]. In addition to capturing the local features of directly adjacent nodes, they can also model the long-range dependencies through the global attention mechanism. **RC-Specific Baselines.** In addition to general-purpose GNNs, we also include several circuit-specific models designed for parasitic-related tasks, including CircuitGPS [37] and CircuitGCL [33] (discussed in Section 2.2).

4 Problem Formulation and Data Analysis

4.1 Multi-level Predictive Tasks

Based on the dataset construction described above, ParasGB further defines diverse circuit parasitic prediction tasks with different granularities, target types, and learning objectives, as summarized in Table 4.

Granularity Variety. The benchmark includes both *node-level* and *edge-level* tasks, each corresponding to distinct physical phenomena. Node-level tasks predict ground capacitance (C_g), a key determinant of circuit timing and power consumption. Edge-level tasks target coupling capacitance (C_c) and effective resistance (R_{eff}), where C_c is critical for analyzing signal cross-talk and noise, and R_{eff} directly impacts signal delay (timing) and voltage drop (IR drop) in power distribution networks.

Objective Variety. We formulate both regression and classification tasks to support different use cases. Regression targets fine-grained numerical estimation, which is useful for parasitic-aware analysis and more detailed early-stage evaluation. Classification

discretizes parasitic values into magnitude bins, which is useful for rapid screening and early-stage design exploration when identifying the order of magnitude is already informative. In addition, coarse-grained categorization of parasitic values may also serve as a useful signal for downstream simplification or partitioning-oriented decisions, analogous to effective resistance-based criteria used in graph reduction and sparsification[25, 51].

4.2 Distribution Analysis and Preprocessing

Label imbalance is a core challenge in industrial circuit data processing. As shown in Figure 3, after applying the task-specific normalization described above, the capacitance and resistance labels still exhibit highly skewed, long-tailed, and in some cases multimodal distributions. In several tasks, most samples cluster within a narrow value range, while only a small fraction of nodes or edges lie in the extreme tails. Such distributions make training particularly difficult, since conventional loss functions tend to fit the dominant modes more easily while underemphasizing rare but critical samples, even though these extremes often have a disproportionate impact on timing, noise, and IR-drop behavior. The specific label distribution can be found in Section A.

Listing 1: Unified ParasGB API for loading and evaluation.

```
from parasgb import RCDataset, Evaluator
# 1) Load dataset
dataset = RCDataset(
    dataset_name='sram',      # or 'analog'
    task_level='node',        # 'node' or 'edge'
    task_type='regression'    # or 'classification'
)
# 2) Build dataloader
loader = dataset.get_dataloader(
    split='train', batch_size=32, shuffle=True
)
# 3) Train model
model = MyModel()
for batch in loader:
    y_pred = model(batch)
    loss = criterion(y_pred, batch.y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
# 4) Evaluate
evaluator = Evaluator(
    dataset_name='sram',
    task='cg_regr'            # e.g., cg_regr, cc_class, r_regr
)
metrics = evaluator.evaluate(y_pred, y_true)
```

Table 6: Performance of Different Models on SRAM Circuits Coupling Capacitance Edge Classification Task

Metric	sram+digtime+timing_ctrl		sandwich		ultra8t		array_128_32_8t	
	Accuracy ↑	F1-Score ↑	Accuracy ↑	F1-Score ↑	Accuracy ↑	F1-Score ↑	Accuracy ↑	F1-Score ↑
GCN	0.7094	0.5395	0.5667	0.4071	0.6154	0.4011	0.6406	0.5361
GAT	0.7010	0.5142	0.5671	0.3847	0.6328	0.4235	0.6942	0.5590
GraphSAGE	0.6735	0.4735	0.5333	0.3685	0.6111	0.3938	0.6182	0.3390
PNA	0.7263	0.5521	0.5772	0.3970	0.6331	0.4293	0.6924	0.5585
SGFormer	0.6550	0.4971	0.5256	0.3981	0.5828	0.4210	0.6213	0.5264
PolyNormer	0.7091	0.5383	0.5795	0.4100	0.6446	0.4233	0.6914	0.5632
CircuitGPS	0.6541	0.5732	0.4584	0.4023	0.6124	0.5370	0.5534	0.4681
CircuitGCL	0.8727	0.7265	0.7673	0.7267	0.7661	0.7121	0.8020	0.7552

Table 7: Performance of Different Models on SRAM Circuits Ground Capacitance Node Classification Task

Metric	sram+digtime+timing_ctrl		sandwich		ultra8t		array_128_32_8t	
	Accuracy ↑	F1-Score ↑	Accuracy ↑	F1-Score ↑	Accuracy ↑	F1-Score ↑	Accuracy ↑	F1-Score ↑
GCN	0.5959	0.5311	0.3661	0.3004	0.3165	0.2930	0.2905	0.2744
GAT	0.6897	0.6165	0.5034	0.4927	0.4582	0.4575	0.4152	0.4149
GraphSAGE	0.6480	0.5799	0.3731	0.3151	0.3634	0.3519	0.2441	0.2149
PNA	0.6511	0.5923	0.4431	0.2619	0.5980	0.3095	0.4983	0.3399
SGFormer	0.6337	0.5508	0.5669	0.5661	0.6718	0.6459	0.4307	0.4287
PolyNormer	0.7808	0.7557	0.5261	0.5256	0.5891	0.5828	0.5608	0.5094
CircuitGPS	0.6031	0.5934	0.4022	0.3315	0.3678	0.2963	0.4012	0.3959
CircuitGCL	0.7088	0.6220	0.5820	0.5819	0.5333	0.5332	0.6757	0.6879

To mitigate these distribution challenges, we adopt a consistent preprocessing framework with task-specific filtering and normalization. Circuit parasitic parameters typically span multiple orders of magnitude, and their numerical scales differ substantially from those of spatial coordinates and device geometries; directly using the raw values can therefore lead to unstable optimization [7]. Before normalization, we remove the sparse physically invalid or numerically unstable labels, including out-of-range capacitance values and zero-valued effective resistances. For SRAM circuits, capacitance labels are transformed using logarithmic scaling, while effective-resistance labels are first mapped to the logarithmic domain and then normalized using the first and ninety-ninth percentiles. For analog circuits, ground-capacitance and effective-resistance labels are normalized through logarithmic maximum-based transformations. All processed labels are clipped to the $[0, 1]$ interval. Regression tasks directly use these normalized continuous values as supervision targets, whereas classification tasks discretize them into five magnitude classes using the predefined boundaries $\{0.2, 0.4, 0.6, 0.8\}$ (details in Appendix C).

5 Experiments

Our implementation is built on PyTorch Geometric (PyG) for graph-based processing [7]. All experiments were conducted on two shared computing clusters equipped with Intel Xeon Silver 4314

CPUs (2.4 GHz), 128 GB system memory, four NVIDIA RTX 4090 GPUs (24 GB VRAM), and three NVIDIA A100 GPUs (40 GB VRAM).

In addition, to ensure reproducibility and lower the barrier to adoption, we provide in the repository the hyperparameter settings, training strategies, and hardware and cluster configurations for all benchmarked models. The ParasGB toolkit follows an OGB-style benchmark interface [12] and is built upon the PyG framework [7], enabling concise and unified access to datasets and evaluation utilities. As illustrated in Listing 1, the toolkit supports the full workflow from data download and preprocessing to standardized evaluation. For large-scale SRAM graphs, it also provides memory-efficient subgraph sampling through NeighborLoader. To ensure fair comparison across methods, we recommend using the unified Evaluator module [12], which automatically handles task-specific metrics. Additional results are available in the repository.

5.1 Performance on SRAM Tasks

We further evaluate all methods on SRAM parasitic classification tasks, including edge-level coupling capacitance classification and node-level ground capacitance classification, as shown in Table 6 and Table 7. The regression experiment is in the Appendix D.2.

The results highlight the effectiveness of circuit-specific models designed for parasitic-related tasks. In particular, for edge-level coupling capacitance classification, CircuitGCL consistently achieves the best results on all four SRAM subsets, obtaining the highest

Table 8: Performance of Different Models on Analog Circuits Effective Resistance Edge Regression Task

Metric	1-4, 6, 8-12, 15-18		5		14		20	
	MAE ↓	R ² ↑	MAE ↓	R ² ↑	MAE ↓	R ² ↑	MAE ↓	R ² ↑
GCN	0.0587	0.8018	0.0637	0.4772	0.1513	-0.5502	0.1687	0.6526
GAT	0.0550	0.8159	0.0717	0.4891	0.1533	-0.6171	0.1871	0.5711
GraphSAGE	0.0461	0.8789	0.0511	0.7451	0.1473	-0.5093	0.1396	0.7057
PNA	0.0439	0.9051	0.0408	0.8565	0.1031	0.2984	0.1463	0.7209
SGFormer	0.0489	0.8721	0.0397	0.8127	0.1224	0.0443	0.1439	0.6474
PolyNormer	0.0407	0.9035	0.0575	0.6137	0.1976	0.3099	0.1314	0.6991
CircuitGPS	0.0769	0.7443	0.0719	0.6890	0.2027	0.5314	0.0992	0.3049
CircuitGCL	0.0606	0.8160	0.0913	0.5706	0.0804	0.7257	0.1793	0.6017

Table 9: Performance of Different Models on Analog Circuits Ground Capacitance Node Regression Task

Metric	1-4, 6, 8-12, 15-18		5		14		20	
	MAE ↓	R ² ↑	MAE ↓	R ² ↑	MAE ↓	R ² ↑	MAE ↓	R ² ↑
GCN	0.0272	0.9570	0.0171	0.4200	0.0216	0.8460	0.0284	0.8432
GAT	0.0367	0.9336	0.0078	0.4879	0.0107	0.8749	0.0230	0.8270
GraphSAGE	0.0473	0.8861	0.0267	0.3098	0.0311	0.8106	0.0371	0.7869
PNA	0.0292	0.9502	0.0089	0.4214	0.0094	0.8507	0.0211	0.8423
SGFormer	0.0682	0.7129	0.0180	0.3159	0.0430	0.5846	0.0267	0.6812
PolyNormer	0.0298	0.9667	0.0155	0.3828	0.0148	0.8638	0.0281	0.7959
CircuitGPS	0.0901	0.7011	0.0379	-0.1327	0.0568	0.5613	0.0415	0.5002
CircuitGCL	0.0305	0.9624	0.0073	0.6164	0.0237	0.8308	0.0193	0.8605

Accuracy and F1-score in every case. This indicates that SRAM coupling-capacitance prediction benefits substantially from circuit-specific inductive bias and from modeling dense local coupling structure. For node-level ground capacitance classification, the results are more heterogeneous across subsets, but circuit-specific modeling remains highly competitive. PolyNormer performs best on the aggregated subset, while CircuitGCL achieves the best results on *sandwich* and *array_128_32_8t*, and SGFormer performs best on *ultra8t*. This suggests that node-level SRAM parasitic patterns vary considerably across designs, and that different models capture different aspects of the task.

Another notable observation is the persistent gap between Accuracy and F1-score, especially for node-level ground capacitance classification. In several subsets, some methods achieve moderate or even relatively high Accuracy, while their F1-scores remain much lower, indicating that label imbalance and long-tail distributions remain major challenges. Overall, these results show that ParasGB provides a challenging benchmark for SRAM parasitic prediction and clearly reveals the performance gains brought by circuit-specific parasitic-related models.

5.2 Performance on Analog Tasks

We further evaluate all methods on analog-circuit parasitic regression tasks, including edge-level effective resistance (R_{eff}) prediction and node-level ground capacitance (C_g) prediction, as shown in

Table 8 and Table 9. The classification experiment is in the Appendix D.3.

Overall, C_g node regression is easier and more stable than R_{eff} edge regression. On the main analog subset (IDs 1–4, 6, 8–12, 15–18), several models achieve strong C_g performance, whereas R_{eff} prediction remains much more challenging, especially on circuit ID 14, where GCN, GAT, and GraphSAGE still yield negative R^2 . This indicates that effective-resistance prediction is more sensitive to circuit-specific structural variation. The results also highlight the value of circuit-specific models designed for parasitic-related tasks. On R_{eff} , PNA achieves the best R^2 on the main subset and on ID 5, while CircuitGCL achieves the best R^2 on IDs 14 and 20. Notably, on the hardest circuit, ID 14, both CircuitGPS and CircuitGCL improve the negative R^2 values observed for several general-purpose GNNs to clearly positive ones, demonstrating clear gains on particularly difficult cases. On C_g , the task is more stable overall: PolyNormer achieves the best R^2 on the main subset, while CircuitGCL performs best on IDs 5 and 20.

Another notable observation is the large variation across individual analog circuits. Model rankings change substantially on IDs 5, 14, and 20, showing that the challenge lies not only in graph size, but also in strong cross-circuit heterogeneity in topology and parasitic patterns. Overall, these results show that ParasGB provides a challenging benchmark for analog parasitic prediction and clearly reveals the benefits of circuit-specific parasitic-related models, especially on hard and poorly predicted cases.

6 Conclusions & Limitations

This paper presents ParasGB, the first open-source benchmark suite for parasitic parameter prediction on circuit graphs. Built from tape-out-proven designs and RC networks extracted by commercial EDA tools, ParasGB provides a physically grounded and standardized platform for pre-layout parasitic modeling. By covering both node-level and edge-level tasks under regression and classification settings, it fills the long-standing gap in public, high-fidelity parasitic benchmarks for reproducible research. Using a unified evaluation protocol and standardized training pipeline, we benchmark diverse GNN architectures and reveal several key challenges of parasitic prediction, including severe label imbalance, long-tailed distributions, and strong structural heterogeneity across circuits. These results show that ParasGB supports not only fair comparison but also the study of model robustness and generalization in parasitic-aware circuit learning.

ParasGB also has several limitations. The current benchmark covers only a limited set of process nodes and analog circuit families. Moreover, as a pre-layout benchmark, it does not directly model detailed placement and routing geometry, nor does it aim to replace full post-layout extraction or signoff flows. Future extensions may broaden process-node coverage, include more diverse and larger analog designs, and further connect parasitic prediction with downstream circuit-analysis tasks.

References

- [1] Zhuomin Chai, Yuxiang Zhao, Yibo Lin, Wei Liu, Runsheng Wang, and Ru Huang. 2022. CircuitNet: an open-source dataset for machine learning applications in electronic design automation (EDA). *Information Sciences* 65, 227401 (2022), 1–227401.
- [2] Dalton Colombo, Felipe Werle, Gilson Wirth, and Sergio Bampi. 2012. A CMOS 25.3 ppm/°C bandgap voltage reference using self-cascode composite transistor. In *2012 IEEE 3rd Latin American Symposium on Circuits and Systems (LASCAS)*. 1–4. doi:10.1109/LASCAS.2012.6180360
- [3] Gabriele Corso, Luca Cavalleri, Dominique Beaini, Pietro Liò, and Petar Velickovic. 2020. Principal neighbourhood aggregation for graph nets. In *Proc. of NeurIPS*. 13260–13271.
- [4] Chenhui Deng, Zichao Yue, and Zhiru Zhang. 2024. Polynormer: Polynomial-Expressive Graph Transformer in Linear Time. In *Proc. of ICLR*.
- [5] Zehao Dong, Weidong Cao, Muhun Zhang, Dacheng Tao, Yixin Chen, and Xuan Zhang. 2023. CKTGNN: CIRCUIT GRAPH NEURAL NETWORK FOR ELECTRONIC DESIGN AUTOMATION. In *Proc. of ICLR*.
- [6] KF Edward. 2009. A low voltage CMOS bandgap reference without using an opamp. In *IEEE Custom Integrated Circuits*, Vol. 13. 501–504.
- [7] Matthias Fey and Jan Eric Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- [8] Jian Gao, Weidong Cao, Junyi Yang, and Xuan Zhang. 2025. AnalogGenie: A Generative Engine for Automatic Discovery of Analog Circuit Topologies. *arXiv preprint arXiv:2503.00205* [cs.LG] doi:10.48550/arXiv.2503.00205
- [9] William L Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Proc. of NeurIPS*. 1025–1035.
- [10] Edward N. Y. Ho and Philip K. T. Mok. 2010. A Capacitor-Less CMOS Active Feedback Low-Dropout Regulator With Slew-Rate Enhancement for Portable On-Chip Application. *IEEE Transactions on Circuits and Systems II: Express Briefs* 57, 2 (2010), 80–84. doi:10.1109/TCSII.2009.2038630
- [11] Edward N. Y. Ho and Philip K. T. Mok. 2010. A chip-area efficient capacitorless CMOS LDO with active feedback and damping zero compensation. In *2010 53rd IEEE International Midwest Symposium on Circuits and Systems*. 574–577. doi:10.1109/MWSCAS.2010.5548891
- [12] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. *arXiv preprint arXiv:2005.00687* (2020).
- [13] Youngwoo Ji, Jungho Lee, Byungsub Kim, Hong-June Park, and Jae-Yoon Sim. 2019. A 192-pW Voltage Reference Generating Bandgap– V_{th} With Process and Temperature Dependence Compensation. *IEEE Journal of Solid-State Circuits* 54, 12 (2019), 3281–3291. doi:10.1109/JSSC.2019.2942356
- [14] Xun Jiang, Yuxiang Zhao, Yibo Lin, Runsheng Wang, Ru Huang, et al. 2024. Circuitnet 2.0: An advanced dataset for promoting machine learning innovations in realistic chip design environment. In *Proc. of ICLR*.
- [15] Nachiket Kapre and André DeHon. 2009. Parallelizing sparse matrix solve for SPICE circuit simulation using FPGAs. In *2009 International Conference on Field-Programmable Technology*. IEEE, 190–198.
- [16] Myungjun Kim and Seonghwan Cho. 2023. An Output-Capacitorless Analog LDO Featuring Frequency Compensation of Four-Stage Amplifier. *IEEE Transactions on Circuits and Systems I: Regular Papers* 70, 2 (2023), 642–654. doi:10.1109/TCSI.2022.3223236
- [17] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *Proc. of ICLR*.
- [18] S. K. Koh and L. Lee. 2014. Low power CMOS bandgap reference circuit. In *2014 IEEE Student Conference on Research and Development*. 1–5. doi:10.1109/SCORED.2014.7072988
- [19] Ka Nang Leung and P.K.T. Mok. 2003. A CMOS voltage reference based on weighted ΔV_{GS} for CMOS low-dropout linear regulators. *IEEE Journal of Solid-State Circuits* 38, 1 (2003), 146–150. doi:10.1109/JSSC.2002.806265
- [20] Ka Nang Leung and Philip KT Mok. 2003. A CMOS voltage reference based on weighted ΔV_{sub} GS/for CMOS low-dropout linear regulators. *IEEE Journal of Solid-State Circuits* 38, 1 (2003), 146–150.
- [21] Guangxiang Li, Huimin Qian, Jianping Guo, Bing Mo, Yan Lu, and Dihui Chen. 2020. Dual Active-Feedback Frequency Compensation for Output-Capacitorless LDO With Transient and Stability Enhancement in 65-nm CMOS. *IEEE Transactions on Power Electronics* 35, 1 (2020), 415–429. doi:10.1109/TPEL.2019.2910557
- [22] Ruguo Li, Yanhan Zeng, Yihan Lin, Jingci Yang, and Hong-zhou Tan. 2020. High-PSR and fast-transient LDO regulator with nested adaptive VFF structure. In *2020 IEEE International Conference on Integrated Circuits, Technologies and Applications (ICTA)*. 51–52. doi:10.1109/ICTA50426.2020.9332114
- [23] Mingjie Liu, William J. Turner, George F. Kokai, Bruce Khailany, David Z. Pan, and Hong Ren. 2021. Parasitic-Aware Analog Circuit Sizing with Graph Neural Networks and Bayesian Optimization. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1372–1377. doi:10.23919/DATE51398.2021.9474253
- [24] Zhiqiang Liu and Wenjian Yu. 2023. Computing effective resistances on large graphs based on approximate inverse of cholesky factor. In *Proc. of DATE*. IEEE, 1–6.
- [25] Zhiqiang Liu and Wenjian Yu. 2023. Computing Effective Resistances on Large Graphs Based on Approximate Inverse of Cholesky Factor. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. doi:10.23919/DATE56975.2023.10137201
- [26] Yan Lu, Yipeng Wang, Quan Pan, Wing-Hung Ki, and C. Patrick Yue. 2015. A Fully-Integrated Low-Dropout Regulator With Full-Spectrum Power Supply Rejection. *IEEE Transactions on Circuits and Systems I: Regular Papers* 62, 3 (2015), 707–716. doi:10.1109/TCSI.2014.2380644
- [27] Yuankai Luo, Lei Shi, and Xiao-Ming Wu. 2024. Classic GNNs are strong baselines: reassessing GNNs for node classification. In *Proc. of NeurIPS*. 97650–97669.
- [28] J. Mahattanakul and J. Chutichatuporn. 2005. Design procedure for two-stage CMOS opamp with flexible noise-power balancing scheme. *IEEE Transactions on Circuits and Systems I: Regular Papers* 52, 8 (2005), 1508–1514. doi:10.1109/TCSI.2005.851395
- [29] Herminio Martínez-García, Antoni Grau-Saldes, and Yolanda Bolea-Monte. 2013. Design of a 4.5-V, 450-mA low-dropout voltage linear regulator based on a cascoded OTA. In *IECON 2013 - 39th Annual Conference of the IEEE Industrial Electronics Society*. 2244–2249. doi:10.1109/IECON.2013.6699480
- [30] Dmitry Osipov and Steffen Paul. 2017. Temperature-Compensated β -Multiplier Current Reference Circuit. *IEEE Transactions on Circuits and Systems II: Express Briefs* 64, 10 (2017), 1162–1166. doi:10.1109/TCSII.2016.2634779
- [31] Sai Surya Kiran Pentapati, Bon Woong Ku, and Sung Kyu Lim. 2021. ML-based wire RC prediction in monolithic 3D ICs with an application to full-chip optimization. In *Proceedings of the 2021 International Symposium on Physical Design*. 75–82.
- [32] Haoxing Ren, George F Kokai, Walker J Turner, and Ting-Sheng Ku. 2020. ParaGraph: Layout parasitics and device parameter prediction using graph neural networks. In *Proc. DAC*. 1–6.
- [33] Shan Shen, Shenglu Hua, Jiajun Zou, Jiawei Liu, Jianwang Zhai, Chuan Shi, and Wenjian Yu. 2025. Transferable Parasitic Estimation via Graph Contrastive Learning and Label Rebalancing in AMS Circuits. In *Proc. of ICCAD*.
- [34] Shan Shen, Xingyang Li, Zhuohua Liu, Yikai Wang, Yiheng Wu, Junhao Ma, Yuquan Sun, and Wei W Xing. 2025. OpenYield: An Open-Source SRAM Yield Analysis and Optimization Benchmark Suite. *arXiv preprint arXiv:2508.04106* (2025).
- [35] Shan Shen, Tianxiang Shao, Xiaojing Shang, Yichen Guo, Ming Ling, Jun Yang, and Longxing Shi. 2019. TS cache: A fast cache with timing-speculation mechanism under low supply voltages. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 28, 1 (2019), 252–262.
- [36] Shan Shen, Hao Xu, Yongliang Zhou, Ming Ling, and Wenjian Yu. 2024. Ultra8T: A sub-threshold 8t sram with leakage detection. *Integration* 98 (2024), 102233.

- [37] Shan Shen, Yibin Zhang, Hector Rodriguez, and Wenjian Yu. 2025. Few-shot Learning on AMS Circuits and Its Application to Parasitic Capacitance Prediction. In *Proc. of DAC*.
- [38] Yichen Shi, Zhuofu Tao, Yuhao Gao, Li Huang, Hongyang Wang, Zhiping Yu, Ting-Jung Lin, and Lei He. 2025. AMSnet 2.0: A Large AMS Database with AI Segmentation for Net Detection. *arXiv preprint arXiv:2505.09155* (2025).
- [39] Pratik Shrestha, Alec Aversa, Saran Phatharodom, and Ioannis Savdis. 2024. EDA-schema: A graph datamodel schema and open dataset for digital design automation. In *Proc. of GLSVLSI*. 69–77.
- [40] Shariq Siddiqui et al. 2021. Composite Interconnects for High-Performance Computing Beyond the 7nm Node. In *2021 IEEE International Electron Devices Meeting (IEDM)*. IEEE, 5–3.
- [41] Min Tan and Qianneng Zhou. 2011. A two-stage amplifier with active miller compensation. In *2011 IEEE International Conference on Anti-Counterfeiting, Security and Identification*. 201–204. doi:10.1109/ASID.2011.5967452
- [42] Yee-Chyan Tan, Harikrishnan Ramiah, S. F. Wan Muhamad Hatta, Chee-Cheow Lim, Rui P. Martins, Pui-In Mak, and Yong Chen. 2025. A Capacitorless Flipped-Voltage-Follower-Based Low-Dropout Regulator Incorporating Adaptive-Compensation Buffer. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 33, 5 (2025), 1422–1426. doi:10.1109/TVLSI.2025.3535630
- [43] Zhuofu Tao, Yichen Shi, Yiyang Huo, Rui Ye, Zixuan Li, Longyu Huang, Chen Wu, Ning Bai, Zitong Yu, Ting-Jung Lin, et al. 2024. AMSNet: Netlist Dataset for AMS Circuits. *arXiv preprint arXiv:2405.09045* (2024).
- [44] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *Proc. of ICLR*.
- [45] Dong Wang, Xiao Liang Tan, and P. K. Chan. 2016. A sub-1V wide-load-range LDO regulator with sustained transient efficiency. In *2016 International Symposium on Integrated Circuits (ISIC)*. 1–4. doi:10.1109/ISICIR.2016.7829748
- [46] Han Wang and Qing Ye. 2006. 0.5-V operational transconductance amplifier for CMOS bandgap reference application. In *2006 8th International Conference on Solid-State and Integrated Circuit Technology Proceedings*. 1705–1707. doi:10.1109/ICSICT.2006.306400
- [47] Qitian Wu, Wentao Zhao, Chenxiao Yang, Hengrui Zhang, Fan Nie, Haitian Jiang, Yatao Bian, and Junchi Yan. 2023. SGFormer: Simplifying and empowering transformers for large-graph representations. *Proc. of NeurIPS* 36 (2023), 64753–64773.
- [48] Jun Yang, Yuyao Kong, Zhen Wang, Yan Liu, Bo Wang, Shouyi Yin, and Longxin Shi. 2019. 24.4 sandwich-RAM: An energy-efficient in-memory BWN architecture with pulse-width modulation. In *Proc. Int. Solid-State Circuits Conf. (ISSCC)*. 394–396.
- [49] Jongho Yoon, Jakang Lee, Donggyu Kim, Junseok Hur, and Seokhyeong Kang. 2025. ParaFormer: A Hybrid Graph Neural Network and Transformer Approach for Pre-Routing Parasitic RC Prediction. In *Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC)*. 513–519. doi:10.1145/3658617.3697599
- [50] Wenjian Yu, Shan Shen, Dingcheng Yang, Haoyuan Li, Jiechen Huang, and Chunyan Pei. 2025. Deep Learning Inspired Capacitance Extraction Techniques. In *Proc. of ASPDAC*. 106–112.
- [51] Xueqian Zhao, Zhuo Feng, and Cheng Zhuo. 2014. An Efficient Spectral Graph Sparsification Approach to Scalable Reduction of Large Flip-Chip Power Grids. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 218–223. doi:10.1109/ICCAD.2014.7001355
- [52] Keren Zhu, Hao Chen, Walker J. Turner, George F. Kokai, Po-Hsuan Wei, David Z. Pan, and Haoxing Ren. 2022. TAG: Learning Circuit Spatial Embedding From Layouts. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. doi:10.1145/3508352.3549384

Appendix

A Dataset Details

A.1 SRAM Features

Table 10: Definition of SRAM Circuit Graph Node Features.

Type	Feature	Definition	Index
Device	M_{mos}	Multiplier of transistors	0
	L	Length of the transistor	1
	W	Width of the transistor	2
	M_{res}	Multiplier of connected resistors	3
	L_{res}	Length of resistor	4
	W_{res}	Width of resistor	5
	M_{cap}	Multiplier of connected capacitor	6
	L_r	Length of capacitor	7
	N_r	Number of capacitor fingers	8
	N_p	Number of ports in the device instance	9
	T	Type code of the device instance	10
Net	N_{mos}	Number of connected transistors	0
	N_g	Number of connected gate terminals	1
	N_{sd}	Number of connected source/drain terminals	2
	N_b	Number of connected base terminals	3
	W_{tot}	Total width of connected transistor	4
	L_{tot}	Total length of connected transistor	5
	N_{cap}	Number of connected capacitors	6
	$L_{r_{tot}}$	Total length of connected capacitors	7
	$N_{r_{tot}}$	Total Number of connected capacitor fingers	8
	N_{res}	Number of connected resistors	9
	$W_{tot,res}$	Total width of connected resistors	10
	$L_{tot,res}$	Total length of connected resistors	11
	N_{port}	Number of connected ports	12
Pin	–	Pin types (G/D/S/B for MOS)	0

For SRAM circuits, we use a statistical aggregated feature set (see Table 10 for details), focusing on the statistical characteristics of the global topology and device distribution [39]. Specific characteristics include: the number of transistors, resistors, capacitors and their multipliers; the number of gates, source drains, and substrates connected to each network cable; and the total width and length of all connectors. These kinds of aggregated features can effectively reduce the feature dimension of ultra-large-scale SRAM arrays, so that the model can robustly capture the global physical characteristics of the circuit at the scale of tens of millions of nodes.

A.2 Datasets Introduction

The benchmarks in this study cover two core areas: SRAM and analog circuits, and the specific characteristics of each subset are as follows.

The core characteristics of SRAM datasets are large-scale and dense topologies. With advanced process technology, the wire spacing inside the storage array is extremely small, resulting in a high density of coupling capacitors. The relevant data is extracted from the real circuit netlist (SPF file) after layout. Given that the node size of SRAM circuits generally reaches more than one million, this dataset can effectively verify the scalability and computational efficiency of the model on large-scale graph structures.

- **ssram:** is an energy-efficient SRAM-based design composed of digital standard cells and SRAM arrays. It is used as a representative small-to-medium-scale memory design for training and validation [35].
- **digtime:** corresponds to a digital/SRAM hybrid clock-generation module for SRAMs. It contains digital cells and SRAM columns and is used to evaluate model generalization on internal SRAM clock-generation logic [35].
- **timing_ctrl:** is a timing-control module composed of standard digital cells that generate control signals for SRAM operations. It provides a digital-control test case with circuit topology different from the training designs.
- **sandwich:** is a sandwich-like memory design in which computational digital circuits and SRAM arrays are organized in an alternating structure, forming a representative large-scale mixed digital-memory design [48].
- **ultra8t:** is a multi-voltage 8T SRAM design containing large analog modules and SRAM arrays. It is used to evaluate the scalability and transferability of graph models on large memory-oriented AMS designs [36].

- **array_128_32_8t**: is a standalone 128-row 32-column 8T SRAM array. It serves as an array-level memory benchmark for evaluating model performance on regular SRAM-array structures.

Unlike SRAM datasets, analog circuits are smaller but have extremely detailed depictions of physical characteristics at the device level. Key parameters such as device channel width W , length L , and distance from the source-drain area to the edge of the isolation tank (LDE effect) are included in the node feature system. The circuit parasitic parameters are extracted through commercial PEX tools. Since the analog circuit is highly sensitive to noise, even a small parasitic parameter prediction error can cause the circuit simulation results to deviate from the expected [50].

- **ID 1**: Low-Voltage Differential Signaling Circuit, It converts a low-frequency reference (5-27 MHz) into a high-frequency clock (100-700 MHz) with minimal phase noise for precision timing applications [20].
- **ID 2**: Operational Amplifier Circuit, it utilizes a quasi-linear temperature characteristic to generate a stable 0.4V reference voltage from an ultra-low supply of only 0.56V, consuming just 4.8 μ A of current [46].
- **ID 3**: Band-Gap Reference Circuit, it generates a stable reference voltage (near the silicon bandgap) by utilizing self-cascode composite transistors and a single resistor, achieving a low temperature coefficient of 25.3 ppm/ $^{\circ}$ C with only 25 μ A of current [2].
- **ID 4**: Band-Gap Reference Circuit, it utilizes resistor-subdivision and resistor-less methods to generate a highly stable 910.88 mV reference voltage with an ultra-low temperature coefficient of 12.99 ppm/ $^{\circ}$ C [18].
- **ID 5**: Low-Dropout Regulator Circuit, it generates a stable reference voltage for LDO regulators by balancing the temperature characteristics of N/P-type MOSFETs, achieving a temperature coefficient of 36.9 ppm/ $^{\circ}$ C with a low supply current of 9.7 μ A [19].
- **ID 6**: Low-Dropout Regulator Circuit, it provides a stable output from a 1.8-4.5V supply with a fast transient response and minimal compensation capacitance (7 pF), supporting up to 100 mA of load current with a low dropout of 0.2 V [10].
- **ID 7**: Low-Dropout Regulator Circuit, it generates a stable reference voltage for LDO regulators by balancing the temperature characteristics of N/P-type MOSFETs, achieving a temperature coefficient of 36.9 ppm/ $^{\circ}$ C with a low supply current of 9.7 μ A [19].
- **ID 8**: Low-Dropout Regulator Circuit, it utilizes an Adaptive-Compensation Buffer (ACB) to dynamically switch between pass transistors, enabling stable operation across a wide load range (0 to 30 mA) without an external capacitor while maintaining a low quiescent current of 6 μ A [42].
- **ID 9**: Low-Dropout Regulator Circuit, it utilizes a tri-loop architecture to achieve an ultra-fast transient response (1.15 ns) and maintains a clean supply with a full-spectrum power supply rejection (PSR > -12 dB up to 20 GHz) while consuming only 50 μ A of quiescent current [26].
- **ID 10**: Operational Amplifier Circuit, it provides a flexible, step-by-step methodology to balance noise performance against power consumption, offering greater design control than previous methods as verified by multi-condition SPICE simulations [28].
- **ID 11**: Low-Dropout Regulator Circuit, it utilizes two parallel active feedback paths to create two pole-zero pairs, providing superior stability and transient response compared to single-path methods while supporting a 100-mA load with only 14 μ A of quiescent current [21].
- **ID 12**: Low-Dropout Regulator Circuit, it utilizes a high-gain, three-stage error amplifier to maintain precise regulation even with an unsaturated pass transistor at ultra-low voltages (0.5 V supply), achieving a state-of-the-art current density of 11.4 A/mm² and a low-frequency PSR of -62 dB [16].
- **ID 13**: Operational Amplifier Circuit, it replaces conventional Miller compensation with an active structure that eliminates the right-half-plane (RHP) zero and introduces a left-half-plane (LHP) zero to cancel the first non-dominant pole, resulting in a 9.4x increase in unity-gain frequency and a significantly smaller compensation capacitor [41].
- **ID 14**: Band-Gap Reference Circuit, it utilizes a combination of four MOSFETs, two lateral PNP transistors, and a well resistor to generate a stable 16 μ A output current with a temperature coefficient of 105 ppm/ $^{\circ}$ C, requiring no external bandgap reference or trimming for process compensation [30].
- **ID 15**: Low-Dropout Regulator Circuit, it utilizes a damping-zero compensation technique and a slew-rate enhancement circuit to achieve stability and fast transients with only 1.5 pF of on-chip capacitance, supporting a 100 mA load with a 200 mV dropout [11].
- **ID 16**: Low-Dropout Regulator Circuit, it utilizes a WCF circuit to maintain a fast transient response and stable voltage regulation across a very wide range of load currents (up to 100 mA) and load capacitances (470 pF to 10 nF), while consuming only 14.4 μ A [45].
- **ID 17**: Low-Dropout Regulator Circuit, it utilizes the nested adaptive FVF structure to achieve an ultra-fast transient response (handling load steps from 1 μ A to 20 mA in just 10 ps) while significantly enhancing PSR (-58.52 dB at 1 MHz) and line regulation [22].
- **ID 18**: Low-Dropout Regulator Circuit, it provides a stable output with high DC gain (101 dB) and a precision bandgap reference to support high-current loads (up to 450 mA) with a 0.5-V dropout, while maintaining solid power supply rejection (54.5 dB at 100 Hz) [29].
- **ID 19**: Band-Gap Reference Circuit, it exploits dimension-dependent effects to cancel out process-induced threshold voltage variations, achieving an ultra-low power consumption of 192 pW and highly stable performance (0.53% process variation) without the need for post-fabrication trimming [13].
- **ID 20**: Band-Gap Reference Circuit, it generates a stable reference voltage using an ultra-low power architecture where most of the 5 μ A current is dedicated to the output, achieving a temperature coefficient of < 10 ppm/ $^{\circ}$ C from a 1 V supply without requiring a high-area operational amplifier [6].

For data preparation, full schematic netlists were parsed to construct graph representations and extract node features based on circuit-level statistics, following the preprocessing pipeline described in [37]. For SRAM circuits, post-layout netlists in Standard Parasitic Format (SPF)

were processed to obtain ground-truth labels for ground capacitance and coupling capacitance. For analog circuits, parasitic extraction (PEX) files were utilized to collect ground-capacitance annotations, while effective resistances were computed directly from the extracted netlists.

A.3 Dataset Labels

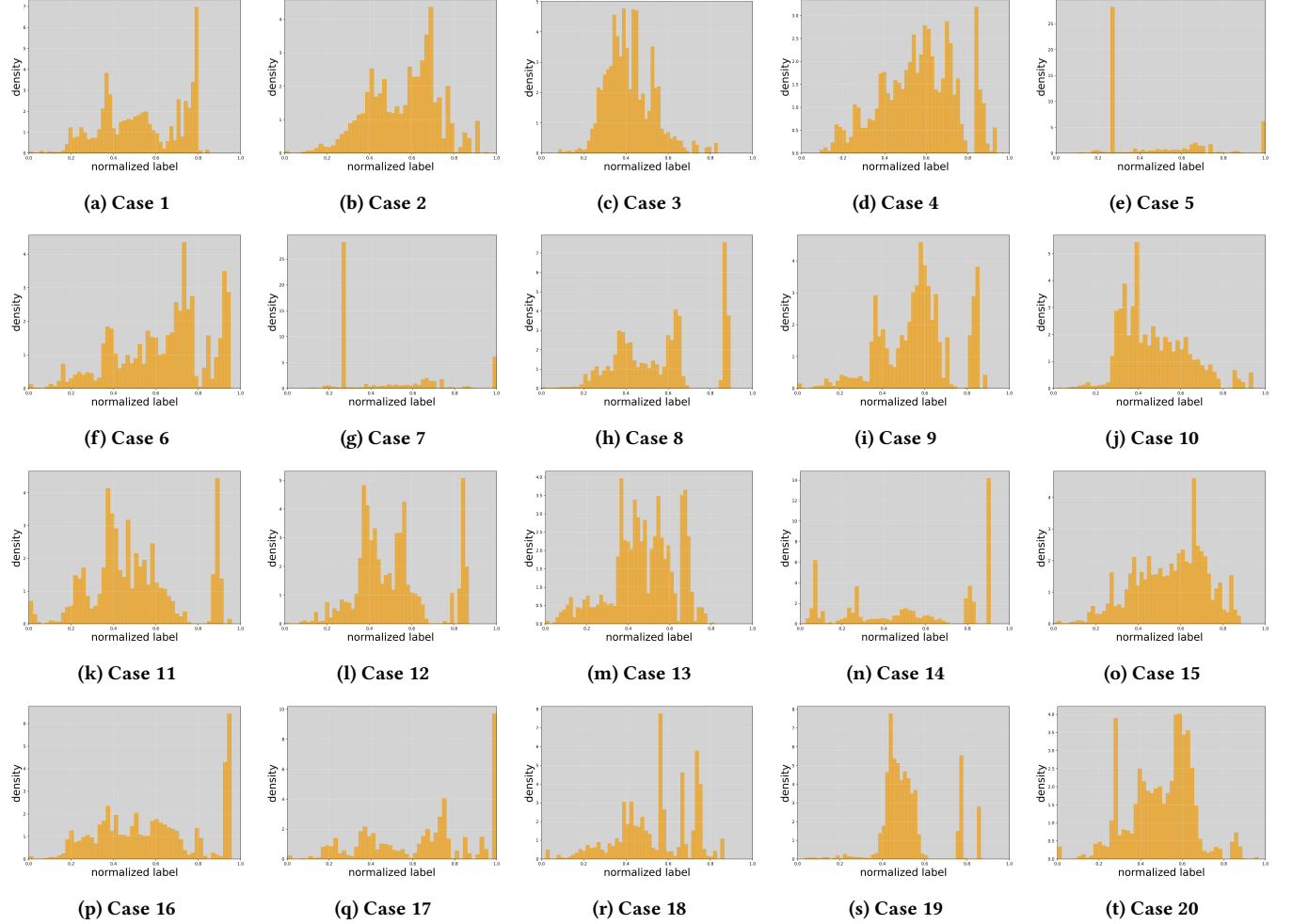


Figure 4: Overview of 20 Analog Circuit Effective Resistance Label Distribution

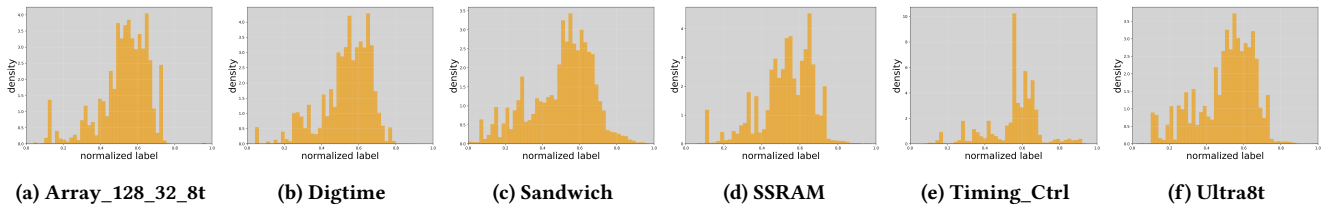


Figure 5: Overview of 6 SRAM Circuit Coupling Capacitance Label Distribution

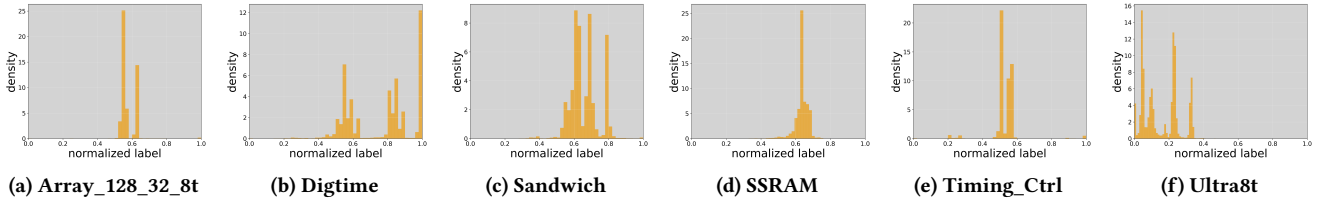


Figure 6: Overview of 6 SRAM Circuit Effective Resistance Label Distribution

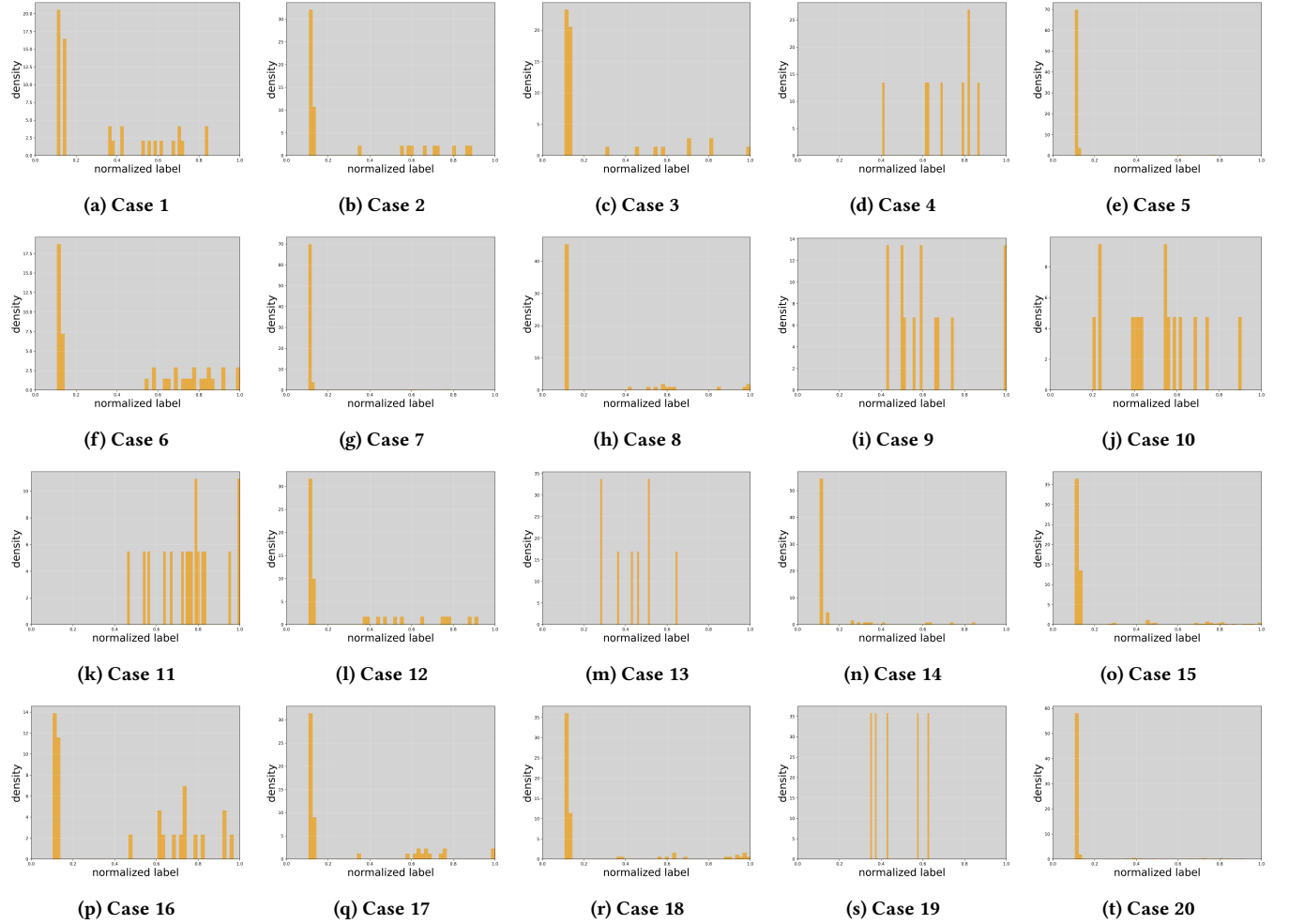


Figure 7: Overview of 20 Analog Circuit Ground Capacitance Label Distribution

This appendix shows the detailed distribution characteristics of the various labels in the dataset. Clarifying the label distribution characteristics is a key prerequisite for training robustness models, and as mentioned in the text, there is a significant label distribution imbalance in circuit data.

Figure 7 shows the distribution characteristics of the 20 analog circuits with ground capacitance (C_g) tags. Despite the differences in the functions of these circuits, their label distribution curves are highly similar, and they all show steep long-tail distribution characteristics, that is, the capacitance values of most nodes are concentrated in a narrow range. Figure 8 shows the ground capacitance distribution characteristics of the 6 core SRAM circuits, covering typical designs such as array_128_32_8t, Sandwich, etc. The label value span of such large-scale circuits is significantly increased, making the regression task challenging [50].

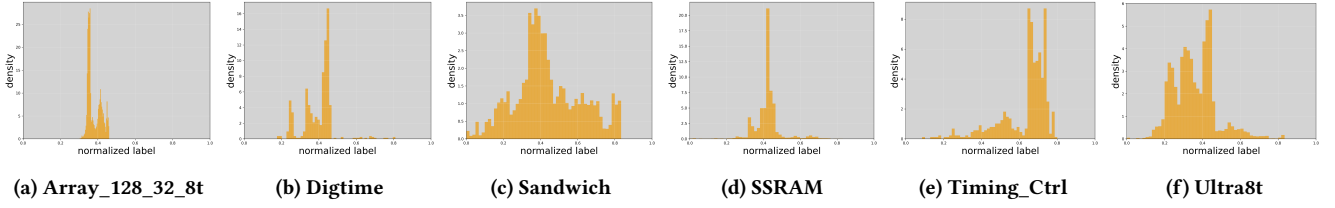


Figure 8: Overview of 6 SRAM Circuit Ground Capacitance Label Distribution

In addition to capacitance, the label distribution characteristics of effective resistors and coupling capacitors were also analyzed. Figure 4 shows the distribution of effective resistors (R) in 20 analog circuits, and the results show that the distribution of resistance labels is more sparse than that of capacitors, and the number of outliers is larger. The core reason for this phenomenon is that the calculation of effective resistance relies on a specific current path, and its physical properties are more complex [39].

Finally, Figure 5 presents the distribution profile of the coupling capacitance (C_c) of the 6 SRAM circuits. The density of wire interconnects is significantly increased under advanced processes, and it can be observed from Figure 5 that the number of coupling capacitors is much higher than that of ground capacitors. Such high-density edge-level labels are the core scenario for verifying the non-local feature capture capability of GNNs [12, 27].

B Analog Topology-To-Graph

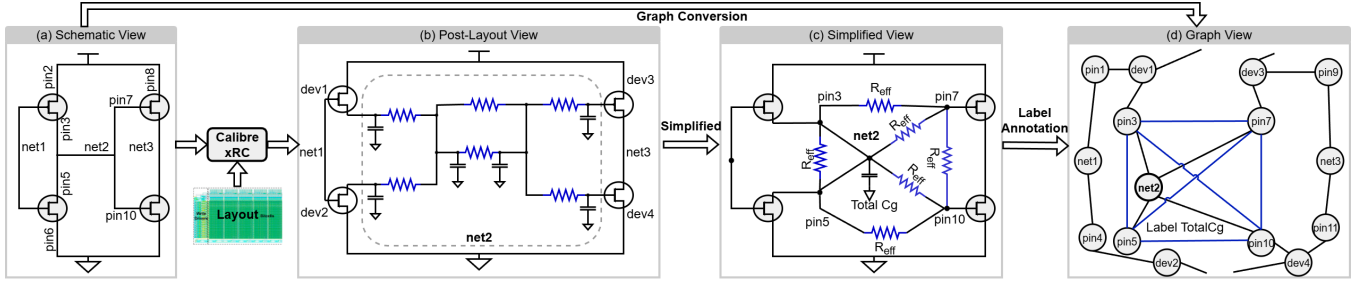


Figure 9: analog circuit graph conversion: (a) The schematic. (b) Post-layout netlist with extracted RCs by StarRC. (c) Our simplified netlist with effective RCs. (d) The converted circuit graph.

The conversion from analog circuit schematics to graph representations follows the framework shown in the figure. We model each circuit as a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. The node set $\mathcal{V}$ contains three types of nodes: *device nodes* representing circuit components, *net nodes* representing interconnect wires, and *pin nodes* representing device terminals. The topological edges $\mathcal{E}_{\text{topo}}$ (shown as black lines) capture circuit connectivity derived from the schematic, specifically through *device-to-pin* and *pin-to-net* connections; these topological relations constitute the input structure obtained from the schematic-to-graph transformation.

In contrast, parasitic information is obtained from the extracted parasitic netlist. Blue *pin-to-pin* edges are treated as resistive edges, where the label corresponds to the effective resistance between two pins (details in Appendix E). In addition, we assign the total ground capacitance of each net as a node-level label on the corresponding net node. These parasitic labels serve as prediction targets in our benchmark.

C Normalization and Label Processing

This section provides the detailed normalization and label construction procedures used in ParasGB. The raw circuit attributes and parasitic labels have substantially different numerical scales, while the capacitance and resistance labels may span several orders of magnitude. We therefore apply task-specific transformations before model training. All data-dependent normalization statistics are computed once from the training split and are subsequently fixed for the validation and test splits. In particular, normalization parameters are not recomputed separately for individual circuits.

C.1 Analog Feature Normalization

For Analog circuits, the continuous attributes of device and net nodes are normalized separately according to their node types. Let $x_{v,j}^{(\tau)}$ denote the j -th feature of node v with node type $\tau \in \{\text{device}, \text{net}\}$. The maximum value of the corresponding feature dimension in the training split is defined as

$$m_{\tau,j} = \max_{v \in V_{\text{train}}^{(\tau)}} x_{v,j}^{(\tau)}. \quad (1)$$

The normalized node feature is then calculated as

$$\hat{x}_{v,j}^{(\tau)} = \frac{x_{v,j}^{(\tau)}}{m_{\tau,j} + \epsilon}, \quad (2)$$

where ϵ is a small positive constant used to avoid division by zero. The normalization is independently performed for each feature dimension and each node type.

C.2 Label Filtering and Normalization

The raw parasitic labels exhibit wide dynamic ranges and highly long-tailed distributions. In addition, a very small fraction of labels have physically abnormal or numerically unstable values. Although these outliers account for only a negligible portion of the dataset, their extreme magnitudes can substantially enlarge the label range, distort the overall distribution, and destabilize model training. We therefore apply task-specific filtering and normalization before constructing the regression and classification targets.

SRAM capacitance labels. Let C_g denote the ground capacitance of a net node and C_c denote the coupling capacitance of a target edge. A capacitance label is considered valid only when

$$10^{-21} < C < 10^{-15}, \quad (3)$$

where C represents either C_g or C_c .

For the edge-level coupling-capacitance task, target edges whose labels fall outside this range are removed:

$$\mathcal{E}_c^{\text{valid}} = \{e \in \mathcal{E}_c \mid 10^{-21} < C_c(e) < 10^{-15}\}. \quad (4)$$

For the node-level ground-capacitance task, directly removing an invalid node would alter the original circuit topology. We therefore preserve the graph structure and replace its invalid label with a negligible placeholder value:

$$C'_g = \begin{cases} C_g, & 10^{-21} < C_g < 10^{-15}, \\ 10^{-30}, & \text{otherwise.} \end{cases} \quad (5)$$

The filtered SRAM capacitance labels are subsequently transformed using logarithmic normalization. The normalized coupling capacitance is defined as

$$\hat{C}_c^{\text{SRAM}} = \text{clip} \left(\frac{\log_{10}(C_c \times 10^{21})}{6}, 0, 1 \right), \quad (6)$$

and the normalized ground capacitance is

$$\hat{C}_g^{\text{SRAM}} = \text{clip} \left(\frac{\log_{10}(C'_g \times 10^{20})}{6}, 0, 1 \right). \quad (7)$$

These transformations compress the large capacitance range while preserving its magnitude ordering.

SRAM effective-resistance labels. Let R_{eff} denote the effective-resistance label of a target edge. Zero-valued labels are removed before normalization:

$$\mathcal{E}_r^{\text{valid}} = \{e \in \mathcal{E}_r \mid R_{\text{eff}}(e) > 0\}. \quad (8)$$

This filtering is required because zero-valued resistance labels are invalid for the considered target pairs and cannot be transformed into the logarithmic domain.

For each remaining label, we first compute

$$z = \log_{10}(R_{\text{eff}}). \quad (9)$$

Let p_1 and p_{99} denote the first and ninety-ninth percentiles of the logarithmic resistance labels in the training data. The normalized effective resistance is defined as

$$\hat{R}_{\text{eff}}^{\text{SRAM}} = \text{clip} \left(\frac{z - p_1}{p_{99} - p_1}, 0, 1 \right). \quad (10)$$

The percentile-based transformation reduces the influence of the sparse extreme values while retaining the relative ordering of most resistance labels.

Analog parasitic labels. The analog circuits have different physical label ranges from the SRAM circuits. Therefore, the fixed capacitance thresholds used for the SRAM subset are not applied to the analog subset. Instead, analog effective-resistance and ground-capacitance labels are normalized using logarithmic maximum-based transformations.

Let

$$M_R = \max_{R_{\text{eff}} \in \mathcal{Y}_{R,\text{train}}} R_{\text{eff}} \quad (11)$$

denote the maximum effective-resistance label in the analog training split. This value corresponds to `edge_labelmax` in the preprocessing implementation. The normalized effective resistance is computed as

$$\hat{R}_{\text{eff}}^{\text{Analog}} = \text{clip} \left(\frac{\ln(1 + R_{\text{eff}})}{\ln(1 + M_R)}, 0, 1 \right). \quad (12)$$

For analog ground capacitance, the raw value is multiplied by 10^{15} so that its numerical magnitude is expressed on the femtofarad scale. We define

$$M_C = \max_{C_g \in \mathcal{Y}_{C,\text{train}}} (10^{15} C_g), \quad (13)$$

which corresponds to `node_labelmax` in the preprocessing implementation. The normalized ground capacitance is then given by

$$\hat{C}_g^{\text{Analog}} = \text{clip} \left(\frac{\ln(1 + 10^{15} C_g)}{\ln(1 + M_C)}, 0, 1 \right). \quad (14)$$

The logarithmic transformations compress the long-tailed analog label distributions while preserving the relative ordering of the original physical values. The $\text{clip}(\cdot, 0, 1)$ operation further restricts all normalized labels to $[0, 1]$ and prevents out-of-range values caused by previously unseen extreme samples.

C.3 Classification Label Construction

The regression tasks directly use the normalized continuous labels defined above as supervision targets. For classification tasks, each normalized label $\hat{y} \in [0, 1]$ is discretized into five magnitude classes using the fixed boundary set

$$\mathcal{B} = \{0.2, 0.4, 0.6, 0.8\}. \quad (15)$$

The corresponding categorical label is defined as

$$y_{\text{class}} = \sum_{b \in \mathcal{B}} \mathbb{I}(\hat{y} > b), \quad (16)$$

where $\mathbb{I}(\cdot)$ denotes the indicator function. The five classes, therefore, correspond to

$$\begin{aligned} C_0 &: [0, 0.2], \\ C_1 &: (0.2, 0.4], \\ C_2 &: (0.4, 0.6], \\ C_3 &: (0.6, 0.8], \\ C_4 &: (0.8, 1]. \end{aligned} \quad (17)$$

This operation is equivalent to applying `torch.bucketize` with the boundaries $[0.2, 0.4, 0.6, 0.8]$ and `right=False`. The same fixed boundaries are used for all circuit designs, ensuring that each class has a consistent numerical interpretation across the complete benchmark. Regression and classification tasks consequently share the same filtered and normalized label space.

D Additional Results

This appendix provides additional results that complement the main experimental analysis. Specifically, we report SRAM regression results and analog classification results, covering both node-level and edge-level parasitic prediction tasks.

D.1 Experimental Settings

The additional experiments follow the same data preprocessing, task definitions, and evaluation protocol as those reported in the main paper.

Data splits. For the SRAM subset, the models are trained and validated on the combined **ssram**, **digtime**, and **timing_ctrl** designs, and evaluated on the unseen **sandwich**, **ultra8t**, and **array_128_32_8t** designs. This split evaluates cross-scale generalization from relatively small training circuits to substantially larger SRAM graphs.

For the analog subset, circuits **1–4**, **6**, **8–12**, and **15–18** are used for training and validation, while circuits **5**, **14**, and **20** are held out for cross-design evaluation. This setting evaluates model generalization across unseen analog circuit topologies and functional categories.

Table 11: Performance of Different Models on SRAM Circuits Ground Capacitance Node Regression Task

Metric	sram+digtime+timing_ctrl		sandwich		ultra8t		array_128_32_8t	
	MAE ↓	R ² ↑	MAE ↓	R ² ↑	MAE ↓	R ² ↑	MAE ↓	R ² ↑
GCN	0.0461	0.8401	0.1876	0.1636	0.0192	0.5446	0.1199	-0.1967
GAT	0.0454	0.8469	0.1838	0.2159	0.0955	0.5060	0.1211	-0.2518
GraphSAGE	0.0413	0.8800	0.1964	0.0585	0.1093	0.2620	0.2813	-2.0157
PNA	0.0415	0.8813	0.1845	0.3589	0.0894	0.4561	0.2024	-0.9577
SGFormer	0.0424	0.8729	0.2297	-0.1733	0.1619	-0.3985	0.2934	-3.2101
PolyNormer	0.0423	0.8667	0.1938	0.1186	0.1563	-0.5618	0.1699	-1.1344
CircuitGPS	0.0063	0.9568	0.0298	0.6574	0.0222	0.7882	0.0194	0.8702
CircuitGCL	0.0412	0.8991	0.2310	0.3726	0.1879	0.5003	0.1678	0.5234

Table 12: Performance of Different Models on SRAM Circuits Coupling Capacitance Edge Regression Task

Metric	sram+digtime+timing_ctrl		sandwich		ultra8t		array_128_32_8t	
	MAE ↓	R ² ↑	MAE ↓	R ² ↑	MAE ↓	R ² ↑	MAE ↓	R ² ↑
GCN	0.0717	0.5008	0.1073	0.3511	0.0971	0.2866	0.0960	0.2543
GAT	0.0717	0.4960	0.1075	0.3504	0.0962	0.2969	0.0956	0.2581
GraphSAGE	0.0747	0.4701	0.1078	0.3427	0.0920	0.3334	0.0783	0.4182
PNA	0.0715	0.5053	0.1067	0.3527	0.0935	0.3083	0.0863	0.3554
SGFormer	0.0766	0.4592	0.1081	0.3212	0.0930	0.3093	0.0746	0.4378
PolyNormer	0.0709	0.5053	0.1053	0.3449	0.0934	0.2951	0.0835	0.3797
CircuitGPS	0.0714	0.4413	0.1093	0.3412	0.0933	0.3284	0.0723	0.3979
CircuitGCL	0.0698	0.5340	0.1124	0.3455	0.0949	0.3375	0.0818	0.4067

Tasks and baselines. The SRAM experiments include ground-capacitance node prediction, coupling-capacitance edge prediction, and effective-resistance edge prediction. The analog experiments include a ground-capacitance node prediction and effective-resistance edge prediction. We evaluate four message-passing GNNs, including GCN, GAT, GraphSAGE, and PNA; two graph Transformers, including SGFormer and PolyNormer, and two circuit-specific methods, including CircuitGPS and CircuitGCL.

Training configuration. For the standard SRAM baselines, we use four GNN layers, two prediction head layers, a hidden dimension of 144, a dropout rate of 0.4, a batch size of 128, and a learning rate of 10^{-4} . Four-hop neighborhoods with at most 64 sampled neighbors per hop are used for mini-batch training. All models are trained for 200 epochs with random seed 42.

For the standard analog baselines, we use four GNN layers, a hidden dimension of 128, a dropout rate of 0.3, a batch size of 64, and a learning rate of 5×10^{-5} . Four-hop neighborhood sampling is used with at most 128 sampled neighbors per hop. For analog edge-level tasks, 60% of the target edges are sampled during training. All analog models are trained for 200 epochs with random seed 42.

CircuitGPS and CircuitGCL use their model-specific configurations. On the SRAM subset, CircuitGPS uses four GNN layers, two prediction head layers, a hidden dimension of 144, a dropout rate of 0.3, and a learning rate of 10^{-4} . CircuitGCL uses four GNN layers, three prediction head layers, a hidden dimension of 96, a dropout rate of 0.4, and a learning rate of 5×10^{-5} . Both models use a batch size of 128.

On the analog subset, CircuitGPS uses four GNN layers, two prediction head layers, a hidden dimension of 84, a dropout rate of 0.4, a learning rate of 10^{-4} , and three-hop subgraph sampling. CircuitGCL uses four GNN layers, two prediction head layers, a hidden dimension of 144, a dropout rate of 0.4, and a learning rate of 10^{-4} .

Evaluation metrics. Regression performance is evaluated using mean absolute error (MAE) and the coefficient of determination (R^2), where lower MAE and higher R^2 indicate better performance. Classification performance is evaluated using accuracy and F1-score.

D.2 Performance on SRAM Regression Tasks

Table 11 and Table 12 report the SRAM ground-capacitance node regression and coupling-capacitance edge regression results. For ground-capacitance regression, CircuitGPS achieves the strongest overall performance, obtaining the best MAE and R^2 on most SRAM test circuits. In particular, it generalizes well from smaller training designs to unseen large-scale SRAM circuits, including *sandwich*, *ultra8t*, and *array_128_32_8t*. However, coupling-capacitance edge regression is more challenging, where the best model varies across circuits and metrics.

Table 13: Performance of Different Models on Analog Circuits Ground Capacitance Node Classification Task

Metric	1-4, 6, 8-12, 15-18		5		14		20	
	Accuracy $\uparrow$	F1-Score $\uparrow$	Accuracy $\uparrow$	F1-Score $\uparrow$	Accuracy $\uparrow$	F1-Score $\uparrow$	Accuracy $\uparrow$	F1-Score $\uparrow$
GCN	0.8718	0.5357	0.9935	0.4727	0.9740	0.3127	0.9101	0.3426
GAT	0.7949	0.3000	0.9941	0.4991	0.9761	0.3239	0.9101	0.3426
GraphSAGE	0.7949	0.3000	0.9946	0.8412	0.9761	0.3322	0.9213	0.5640
PNA	0.8974	0.5804	0.9941	0.7028	0.9783	0.5189	0.9326	0.7926
SGFormer	0.8462	0.4375	0.9941	0.4991	0.9783	0.4122	0.9101	0.3426
PolyNormer	0.8974	0.6476	0.9941	0.7028	0.9783	0.4922	0.9213	0.5640
CircuitGPS	0.8460	0.3806	0.9911	0.2492	0.9740	0.3215	0.8876	0.2714
CircuitGCL	0.8966	0.9138	0.9855	0.3318	0.9681	0.3989	0.8981	0.3527

Table 14: Performance of Different Models on Analog Circuits Effective Resistance Edge Classification Task

Metric	1-4, 6, 8-12, 15-18		5		14		20	
	Accuracy $\uparrow$	F1-Score $\uparrow$	Accuracy $\uparrow$	F1-Score $\uparrow$	Accuracy $\uparrow$	F1-Score $\uparrow$	Accuracy $\uparrow$	F1-Score $\uparrow$
GCN	0.5314	0.4462	0.8474	0.4678	0.5007	0.2875	0.3316	0.3323
GAT	0.5216	0.4240	0.8018	0.3424	0.2735	0.1651	0.1634	0.1255
GraphSAGE	0.5164	0.4238	0.8415	0.4505	0.3934	0.2437	0.3381	0.3395
PNA	0.4649	0.2971	0.8132	0.3918	0.3058	0.1995	0.1060	0.0809
SGFormer	0.5199	0.5163	0.8684	0.5575	0.4024	0.3316	0.3638	0.3618
PolyNormer	0.5017	0.4229	0.2636	0.1747	0.3565	0.2049	0.1859	0.2265
CircuitGPS	0.6491	0.6415	0.6851	0.3573	0.2978	0.2778	0.3360	0.2386
CircuitGCL	0.7018	0.7339	0.7837	0.4491	0.6189	0.5545	0.3265	0.2472

CircuitGCL achieves the best performance on *sram+digtime+timing_ctrl* and the best R^2 on *ultra8t*, while PolyNormer, GraphSAGE, CircuitGPS, PNA, and SGFormer achieve the best scores on different remaining columns. These results indicate that SRAM edge-level coupling prediction is more difficult than node-level ground-capacitance prediction, especially under cross-scale generalization.

D.3 Performance on Analog Classification Tasks

Table 13 and Table 14 report the analog ground-capacitance node classification and effective-resistance edge classification results. For ground-capacitance classification, PNA and PolyNormer achieve the best accuracy on the main analog subset, while CircuitGCL obtains the best F1-score, suggesting that imbalance-aware learning improves minority-class recognition. On held-out analog circuits **5**, **14**, and **20**, the best model varies across circuit IDs, reflecting the structural diversity of analog designs. For effective resistance classification, no single model dominates all test circuits. CircuitGCL performs best on the main analog subset and achieves the highest F1-score on circuit **14**, while SGFormer performs best on circuits **5** and **20**. These results further show that analog classification is sensitive to cross-design structural shifts and benefits from both circuit-specific modeling and imbalance-aware training.

E Algorithms

To generate high-precision physical labels for the dataset, the effective resistance (R) between any two nodes in the circuit graph is calculated. The time cost of direct circuit simulation is high, so this study adopts an efficient calculation method based on matrix operations.

E.1 Matrix-Based Effective Resistance Calculation

The core principle of this method is to use the node admittance matrix of the circuit to realize the effective resistance calculation. As shown in Algorithm 1, first fill all the resistance information in the circuit netlist to the node admittance matrix G , and each row and column of the matrix corresponds to a node in the circuit.

In the first stage of the algorithm, all the original resistive elements are traversed, the resistance value is converted to a conductance value ($g = 1/r$), and the diagonal and non-diagonal elements of the matrix G are updated according to Kirchhoff’s law of current. Then the

Algorithm 1 Matrix-Based Effective Resistance Calculation

Input: Resistor List $\mathcal{R}$ of a Net, Port List $\mathcal{P}$
Output: List of effective resistances $\mathcal{L}_{out} = \{(src, dst, val)\}$
Initialization
 $\mathcal{L}_{out} \leftarrow \emptyset$
 $\mathcal{V} \leftarrow \text{ExtractUniqueNodes}(\mathcal{R})$
 $N \leftarrow |\mathcal{V}|$
if $N < 2$ **or** $|\mathcal{P}| < 2$ **then**
 return $\emptyset$
end if
 $\mathcal{M} \leftarrow \text{MapNodesToIndices}(\mathcal{V})$

Stage 1: Construction of Invertible Admittance Matrix
 $G \leftarrow \mathbf{0}_{N \times N}$
for all $(n_1, n_2, r) \in \mathcal{R}$ **do**
 $g \leftarrow 1/r$
 $u \leftarrow \mathcal{M}[n_1], \quad v \leftarrow \mathcal{M}[n_2]$
 $G_{u,u} \leftarrow G_{u,u} + g, \quad G_{v,v} \leftarrow G_{v,v} + g$
 $G_{u,v} \leftarrow G_{u,v} - g, \quad G_{v,u} \leftarrow G_{v,u} - g$
end for
 $ref \leftarrow N - 1$ // Set last node as reference
 $G_{red} \leftarrow G[0 : ref, 0 : ref]$

Stage 2: Computing the Inverse of Cholesky Factor and Port-to-Port Resistances
// Compute the Cholesky factor L : $G_{red} = LL^T$
Compute the Cholesky factor L from G_{red} , where L is a lower-triangular matrix
 $Z \leftarrow L^{-1}$ // Compute L 's inverse
for $k = 0$ **to** $|\mathcal{P}| - 1$ **do**
 for $l = k + 1$ **to** $|\mathcal{P}| - 1$ **do**
 $src_id \leftarrow \mathcal{P}[k]$
 $dst_id \leftarrow \mathcal{P}[l]$
 $z_{src} \leftarrow$ the $\mathcal{M}[src_id]$ -th column of Z
 $z_{dst} \leftarrow$ the $\mathcal{M}[dst_id]$ -th column of Z
 $Req \leftarrow \|z_{src} - z_{dst}\|^2$
 $\mathcal{L}_{out}.add((src_id, dst_id, Req))$
 end for
end for
return $\mathcal{L}_{out}$

reference node (usually the ground node) is selected, and the corresponding matrix row/column of the node is eliminated to obtain the invertible admittance matrix.

The second stage of the algorithm is the core step, i.e., using the inverse of the Cholesky factor of the invertible admittance matrix to obtain the effective resistances. The inverse of the Cholesky factor effectively captures the inverse of the admittance matrix. And with it, one can easily query the effective resistance between any two nodes in the circuit [24]. The computational efficiency of this method is significantly better than that of the traditional path search method, and can efficiently support the generation of tens of millions of edge-level labels in the dataset.